

Front_end q4 notes

一. JavaScript和ES6的学习

首先我学习了JavaScript和ES6的相关知识，重点学习了JS的变量作用域，闭包函数，对象，原型链，异步编程和ES6的一些新特性，现将重要的学习过程总结如下。

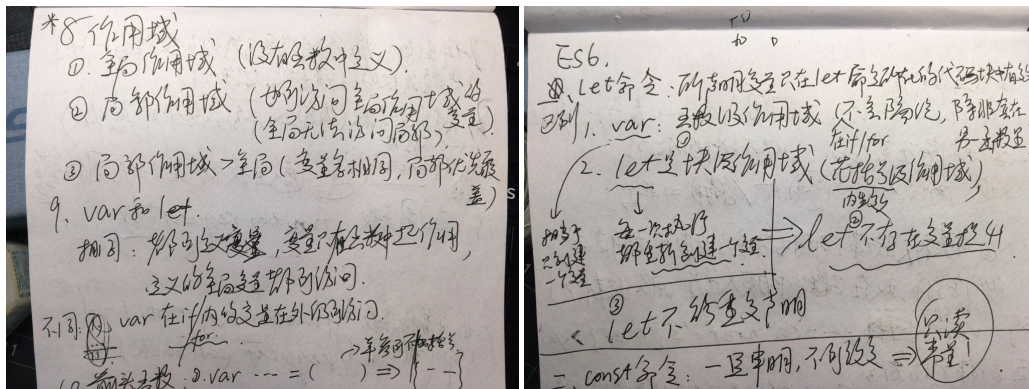
• 作用域与不同变量声明的区别

作用域分全局作用域（整个程序都可访问）和局部作用域（代码块中可以访问）两大类。

var：不是纯粹的全局作用域，需要分情况讨论（在函数内部声明是函数作用域，在全局声明是全局作用域，在代码块内部声明会泄漏到函数/全局作用域）；

let：是一种块级作用域，不能进行变量提升，在全局作用域中无法声明全局变量；

（**const**：声明的变量必须赋值（即只读常量），和let类似不能进行变量提升）

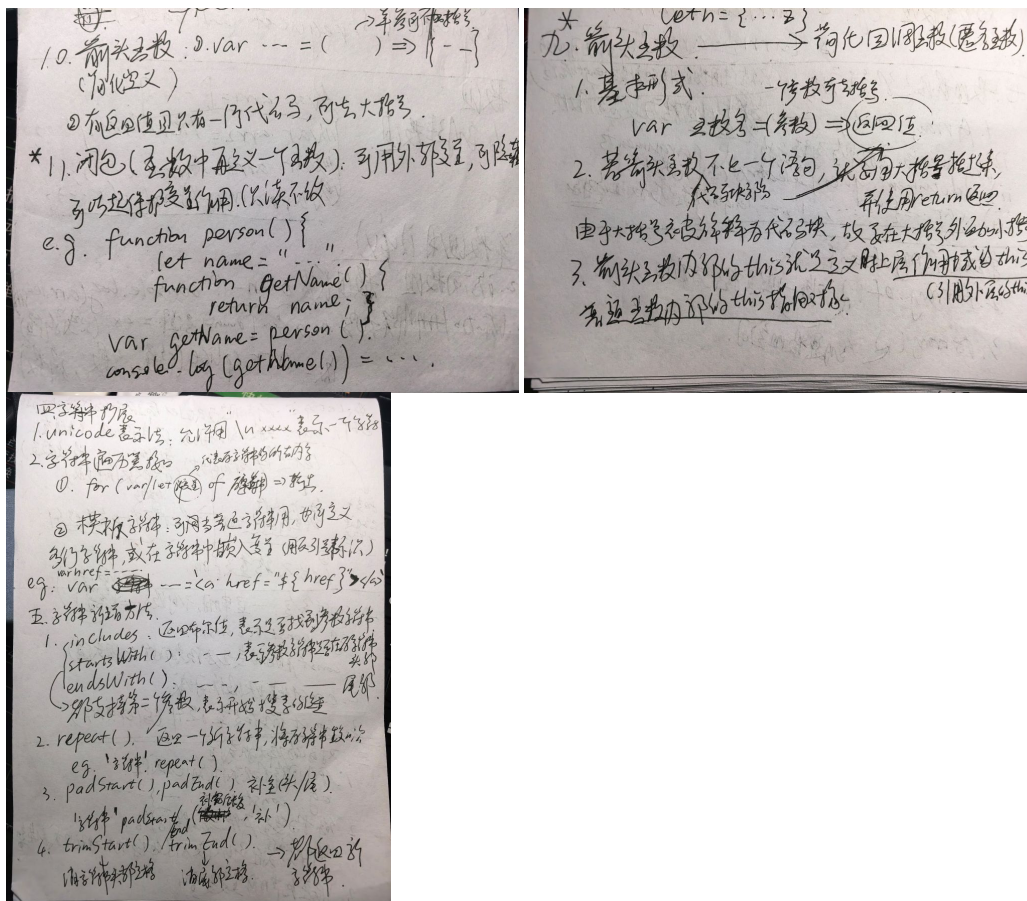


• 函数扩展：闭包，箭头函数，模版字符串

闭包：函数能够记住并访问其词法作用域，并能进行数据封装，创建私有变量（多用let声明）；

箭头函数：是一种简洁的函数写法，会自动绑定外层this（故箭头函数不适合作为对象的方法）；

模版字符串：使用反引号创建多行字符串和字符串插值，使得字符串拼接更直观；



- 对象：基本概念，set数据结构，原型链，异步编程，对象的解构赋值

对象：键值对 (key&value) 集合，JavaScript 的核心；

set数据结构：值唯一的集合，高效去重和查找；

原型链：JavaScript 的继承机制，对象通过原型链共享属性和方法，实现代码复用；

异步编程：处理耗时操作，从回调 $\rightarrow$ Promise $\rightarrow$ async/await 演进；

解构赋值：从数组或对象中提取值到变量中，使得提取数据更简洁，代码更清晰；

1. 创建对象 (对象与列值) $\Rightarrow$ key: value
 eg. var employee = {
 name: "...",
 ...
 }
 2. 对象属性 (通过属性)
 console.log (employee);
 3. 对象key: 对每个属性设置, 且key与值对应
 eg. var name = "lax";
 employee.name = name;
 console.log (employee);
 4. 遍历对象属性
 console.log (Object.keys (employee));
 5. for (key in employee) { console.log (key); }

1. 对象与属性
 1. 属性名与属性值同名, 则值被写入属性/函数
 2. 属性名与属性值 (直接写函数名)
 函数名 { ... }
 3. 属性名与属性值 = 用属性名作为属性值
 (放在括号内)
 4. 对象与属性
 eg. let z = { a: 3, b: 4 }
 let n = { ... z }

* 获取对象属性, Object.getOwnPropertyNames
 1. 原型链: 每个对象都有一个原型链指向其原型
 查找 object.getPrototypeOf ()
 2. 修改原型链: 修改原型链
 Object.setPrototypeOf ()
 3. spread 操作符 (与 rest 操作符类似, 用于数组)
 { ... , spread } spreadClone
 或 var ... , spread

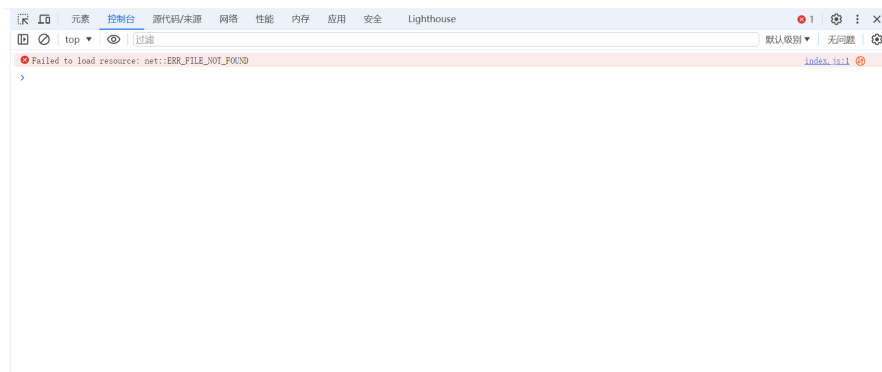
1. setTimeout () (延迟执行)
 set Timeout () $\Rightarrow$ { c.l () } 1000
 2. setInterval () (每隔一段时间执行一次)
 set Interval () { }
 3. 创建 Promise
 eg. var promise = new Promise (resolve $\Rightarrow$ {
 set Timeout () { ... } })
 4. promise 交互回调
 then ()
 promise.then (value $\Rightarrow$ c.l (value))
 5. promise 异常捕获
 catch ()
 promise.catch (error $\Rightarrow$ c.l (error))

1. Promise
 1. Promise 是一个构造函数, 用于生成 promise 对象
 eg. const promise = new Promise (function (resolve, reject) {
 if () { resolve (value); }
 else { reject (error); }
 })
 Promise 构造函数接收一个函数作为参数, 函数接收 resolve 与 reject 两个参数, 用于生成 promise 对象
 Promise 对象的状态, 分为 resolved 状态与 rejected 状态
 promise.then (function (value) { ... }, function (error) { ... })
 2. Promise 交互: Ajax 请求
 Promise 对象是 Ajax 请求的异步操作
 3. async 函数: 异步函数
 eg. function timeout (ms) {
 const promise = new Promise (resolve, reject)
 => { set Timeout (resolve, ms) }
 (async function asyncPrint (value, ms) {
 await timeout (ms); c.l (value);
 })

1. Set 数据结构
 1. Set 是一个构造函数, 用于生成 Set 对象
 eg. var s = new Set ();
 var arr = [10, 20, 30];
 arr.forEach (x $\Rightarrow$ s.add (x));
 for (let i of s) { c.l (i); }
 Set 结构不会加入重复的值
 2. Set 结构接收数组作为参数
 eg. var arr = [10, 20, 30, 40];
 var s = new Set (arr);
 c.l ([... s]);
 3. Set 结构不会发生类型转换
 4. size 属性: 返回 Set 结构的成员个数 (即: 长度)
 5. Set 结构的操作符
 1. s.add () 添加数据
 2. s.delete () 删除数据
 3. s.has () 返回布尔值, 表示该元素是否在 Set 结构中
 4. s.clear () 清除 Set 结构

二.做题过程中的问题及尝试

首先, 打开index.html页面, 发现初始显示空白, 开发者工具的控制台如图所示。



造成这种情况的原因是函数未补全，html文件引用的js文件不完全，需要补全函数才能显示；

（考古：这里其实是因为电脑上直接打开的是html文件绝对路径，在控制台中找不到js文件）

观察题目结构，发现了orderStudentGrade函数需要返回一个按小组分组且排序后的对象，这个对象中包含三个数组，每一个数组中是按总成绩降序排列的学生，故而整体思路就是这样的：首先为每个学生计算总成绩（数组的遍历），然后按小组分组学生，对每组内的学生按总成绩降序排序（遍历加排序），最后返回分组排序后的对象。

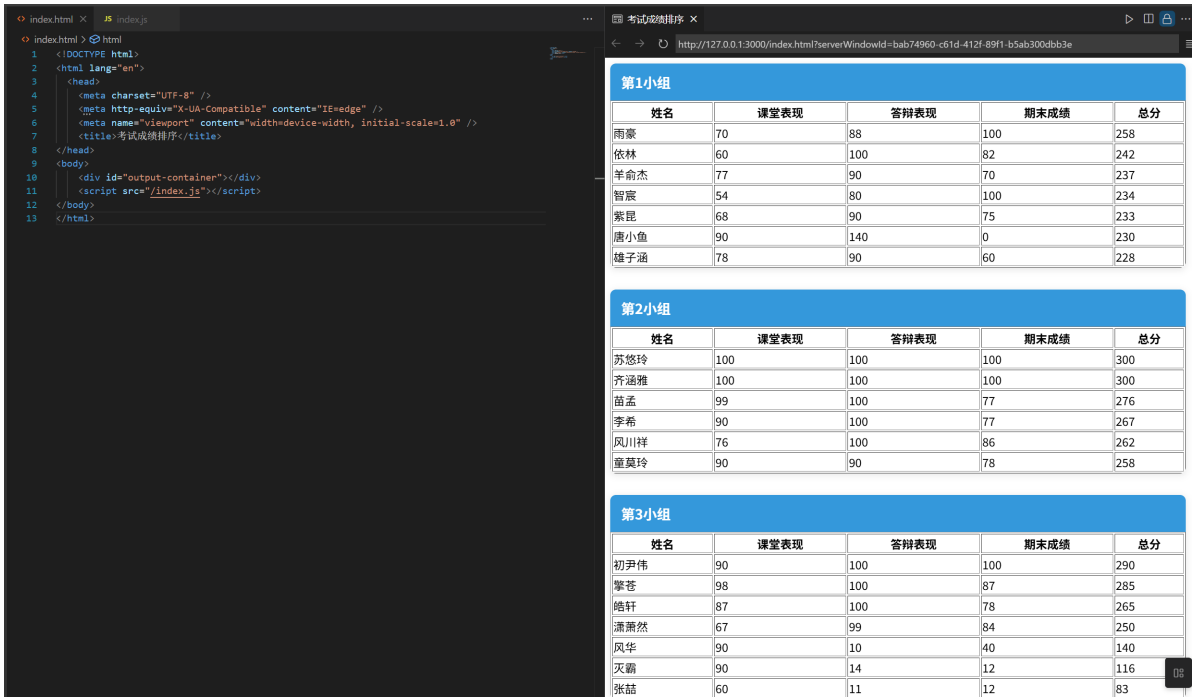
第一步，使用const声明变量studentsTotal使它被赋上对象students，利用Array.map()方法遍历students的每一个属性，再用const声明total为总分，最后返回studentTotal（新的students），里面的每一个子对象都多了一个total的属性；

第二步，先创建一个空对象groupClass，用于按小组分组存储学生数据。再用forEach遍历所有学生，用if条件判断该组是否已存在；如果不存在，创建空数组，将学生添加到对应小组的数组中，这样就将所有学生（studentTotal下的子对象）都分装到了三个数组中（分别代表三个小组）；

第三步，使用Array.sort()方法来对每组内的学生按总成绩降序排序，并返回最终的对象groupClass。

```
1 function orderStudentGrade(students) {  
2   // 计算每个学生的总分  
3   const studentsTotal = students.map(student => {  
4     const total = student.classPerformance + student.defense + student.finalExam ;  
5     return {  
6       ...student,  
7       total,  
8     };  
9   });  
10  // 按小组分组  
11  const groupClass = {};  
12  studentsTotal.forEach(student => {  
13    if (!groupClass[student.class]) {  
14      groupClass[student.class] = [];  
15    }  
16    groupClass[student.class].push(student);  
17  });  
18  // 对每组内的学生按总成绩降序排序  
19  Object.keys(groupClass).forEach(cls => {  
20    groupClass[cls].sort((a, b) => b.total - a.total);  
21  });  
22  return groupClass;  
23 }  
24 const output = document.getElementById("output-container");
```

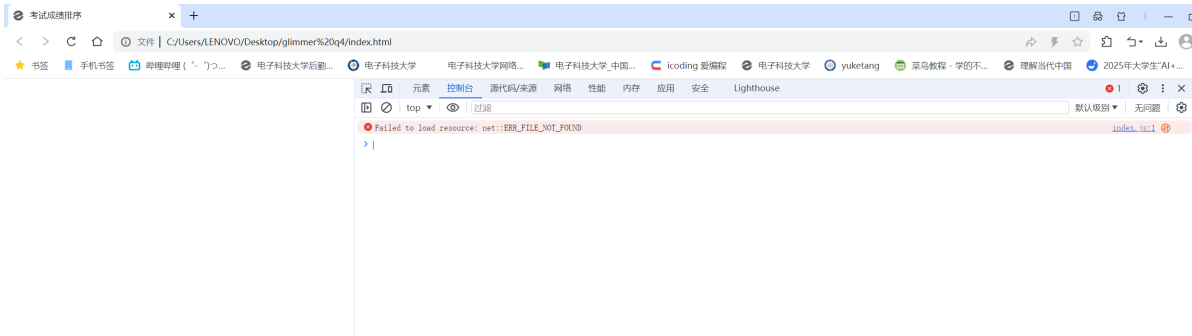
最后使用vs自带的文件预览以及通过vs打开浏览器，都能看到最后呈现的效果。



第1小组				
姓名	课堂表现	答辩表现	期末成绩	总分
雨豪	70	88	100	258
依林	60	100	82	242
羊俞杰	77	90	70	237
智宸	54	80	100	234
紫昆	68	90	75	233
唐小鱼	90	140	0	230
雄子涵	78	90	60	228

第2小组				
姓名	课堂表现	答辩表现	期末成绩	总分
苏悠玲	100	100	100	300
齐涵雅	100	100	100	300
苗孟	99	100	77	276
李希	90	100	77	267
风川祥	76	100	86	262
童莫玲	90	90	78	258
第3小组				
姓名	课堂表现	答辩表现	期末成绩	总分
初尹伟	90	100	100	290
擎苍	98	100	87	285
皓轩	87	100	78	265
潘萧然	67	99	84	250
风华	90	10	40	140
灭霸	90	14	12	116
张喆	60	11	12	83

但是奇怪的是，当我在文件夹中直接打开index.html时，浏览器就无法显示了，控制台报错显示为找不到文件位置。观察之后，我发现应该是因为文件打开的路径不同导致的。文件夹中直接打开的路径就是存放在电脑中的路径，而vs中打开的路径长得像一个域名（？）



在寻求帮助和上网查询后，我得知这是一个有关相对路径和绝对路径的问题。绝对路径（Absolute path）是目标文件在电脑硬盘上的真实路径，从根目录开始；而相对路径（Relative path）是目标文件相对于当前文件的路径，从当前目录开始。绝对路径一般适合脚本，程序，链接等需要可靠定位的地方，而相对路径虽然容易因目录改变而出错，但是可以在项目内部引用资源，便于整体移动（比如远程传输）。

例如一开始使用的vs插件liveserver相当于把文件部署到本地服务器上，以服务器为根目录，则在html文件中使用src="/index.js"相对路径可以访问到这个js文件（两者在同一个根目录下）；若在电脑文件夹中点开则是直接引用html文件中的绝对路径，访问不到相对路径下的js文件。

我还发现，先前包括现在我在markdown中插入的图片使用的都是绝对路径，这就导致了在md文件导出为html文件并上传到github后，html文件无法找到图片的绝对路径，故而会显示不出图片。

于是我马上付诸实践。我将招新第一题的图片路径在html文件中由绝对路径改为相对路径，原本在vs预览上不可显示的图片显示了出来，同理也能够在远程传输后正常显示出图片了。

